

# 15-442/15-642: Machine Learning Systems

## GEMM on Modern GPUs

Spring 2026

Tianqi Chen and Zhihao Jia

Carnegie Mellon University



# Outline

Swizzle Deeper Dive

Tensor Cores

TMA (Tensor Memory Accelerator)

Putting It Together

# Outline

Swizzle Deeper Dive

Tensor Cores

TMA (Tensor Memory Accelerator)

Putting It Together

# Recap: Simple 8x8 Swizzle

Swizzle Layout — 8×8

mapped\_col = logical\_col XOR row | 8×8 matrix, 1 element = 1 bank

Read 

Read Col

Read Row

 Index 

None

0

1

2

3

4

5

6

7

Without Swizzle

|    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|
|    | b0 | b1 | b2 | b3 | b4 | b5 | b6 | b7 |
| r0 | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| r1 | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| r2 | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| r3 | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| r4 | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| r5 | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| r6 | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| r7 | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  |

Bank Activity

B0

B1

B2

B3

B4

B5

B6

B7

With Swizzle (XOR)

|    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|
|    | b0 | b1 | b2 | b3 | b4 | b5 | b6 | b7 |
| r0 | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| r1 | 1  | 0  | 3  | 2  | 5  | 4  | 7  | 6  |
| r2 | 2  | 3  | 0  | 1  | 6  | 7  | 4  | 5  |
| r3 | 3  | 2  | 1  | 0  | 7  | 6  | 5  | 4  |
| r4 | 4  | 5  | 6  | 7  | 0  | 1  | 2  | 3  |
| r5 | 5  | 4  | 7  | 6  | 1  | 0  | 3  | 2  |
| r6 | 6  | 7  | 4  | 5  | 2  | 3  | 0  | 1  |
| r7 | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |

Bank Activity

B0

B1

B2

B3

B4

B5

B6

B7

Bank 0

Bank 1

Bank 2

Bank 3

Bank 4

Bank 5

Bank 6

Bank 7

Current read

Earlier cycle

Not read

Cell value = logical column | Click cell to show mapped location

Cycle-by-Cycle Reads

Without Swizzle

Select a read index to see cycles

With Swizzle (XOR)

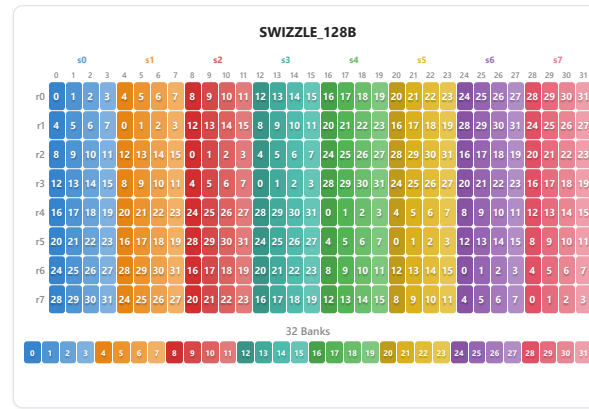
Select a read index to see cycles

# Real Example: SWIZZLE\_128B

## Swizzle Layout — SWIZZLE\_128B

```
physical_sector = logical_sector XOR row | 8 rows x 32 banks, 4 banks per sector
```

|      |                   |                 |       |      |      |      |       |        |        |        |        |        |
|------|-------------------|-----------------|-------|------|------|------|-------|--------|--------|--------|--------|--------|
| Read | Column Tile (8×4) | Row Tile (1×32) | Index | None | c0–3 | c4–7 | c8–11 | c12–15 | c16–19 | c20–23 | c24–27 | c28–31 |
|------|-------------------|-----------------|-------|------|------|------|-------|--------|--------|--------|--------|--------|



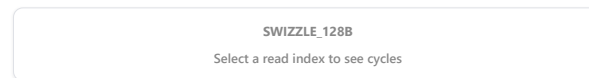
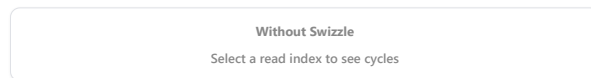
■ ■ ■ s0 
 ■ ■ ■ s1 
 ■ ■ ■ s2 
 ■ ■ ■ s3 
 ■ ■ ■ s4 
 ■ ■ ■ s5 
 ■ ■ ■ s6 
 ■ ■ ■ s7

Current cycle   Earlier cycle   Not read

Cell = logical bank col

Click cell to show mapped location

### Cycle-by-Cycle Reads



1 element = 1 bank (4 bytes). A warp of 32 threads reads 32 elements; bank conflicts serialize into extra cycles.

**Column Tile (8x4):** 8 rows x 4 cols (1 sector) = 32 reads. With swizzle: **1 cycle**. Without: **8 cycles** (8-way conflicts).

**Row Tile (1×32):** 1 row × 32 cols = 32 reads. Always **1 cycle** (all 32 banks distinct within a row).

# SWIZZLE\_128B: Bank Sector View

## Swizzle Atom — SWIZZLE\_128B (8×8)

8 rows, 8 sector columns, 8 bank sectors total (note: we are just collapsing all banks in a sector into a single cell)

Atom = 8 rows × 8 cols = 8 × 128B

swizzled\_col = logical\_col XOR row

Read

Col Tile (8×1)

Row Tile (1×8)

Index

None

0

1

2

3

4

5

6

7

dtype

f8

f16

f32

1 cell = 8 f16

Atom = 8×64 f16

Read tile = 8×8 f16



### Cycle-by-Cycle Reads

Without Swizzle

Select a read index to see cycles

With Swizzle (XOR)

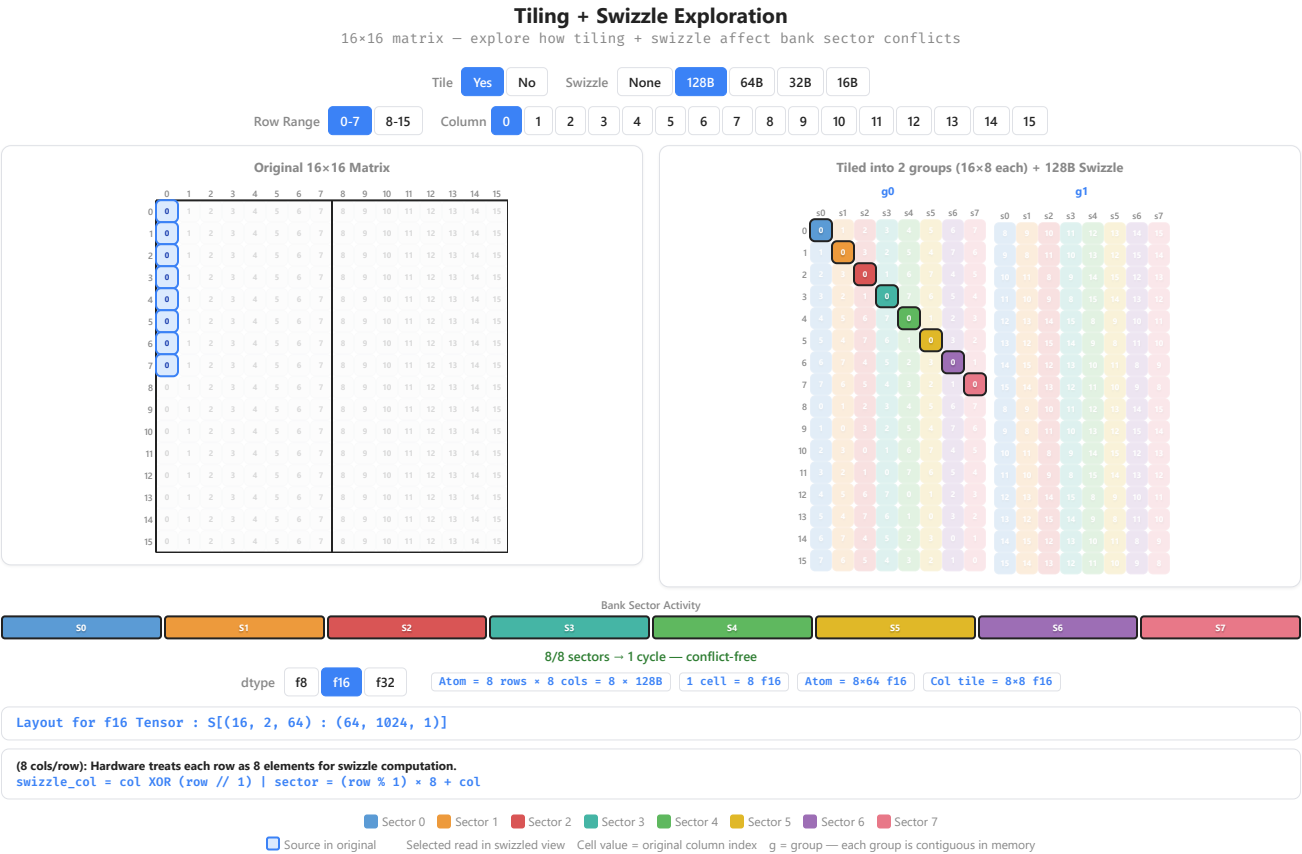
Select a read index to see cycles

# Swizzle Atoms: All Formats





# Swizzle Imposes Tiling Constraint



# Outline

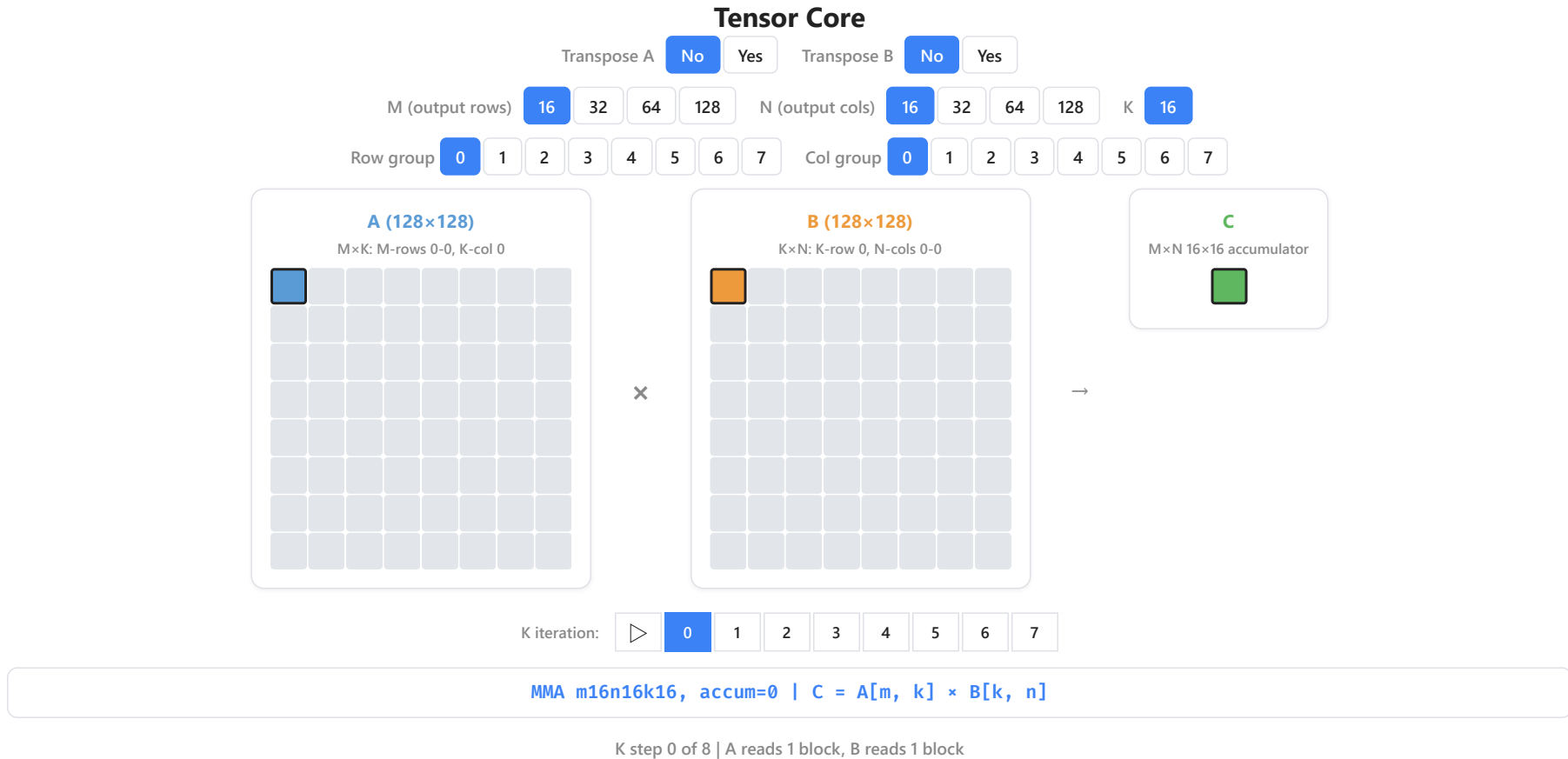
Swizzle Deeper Dive

Tensor Cores

TMA (Tensor Memory Accelerator)

Putting It Together

# Tensor Core

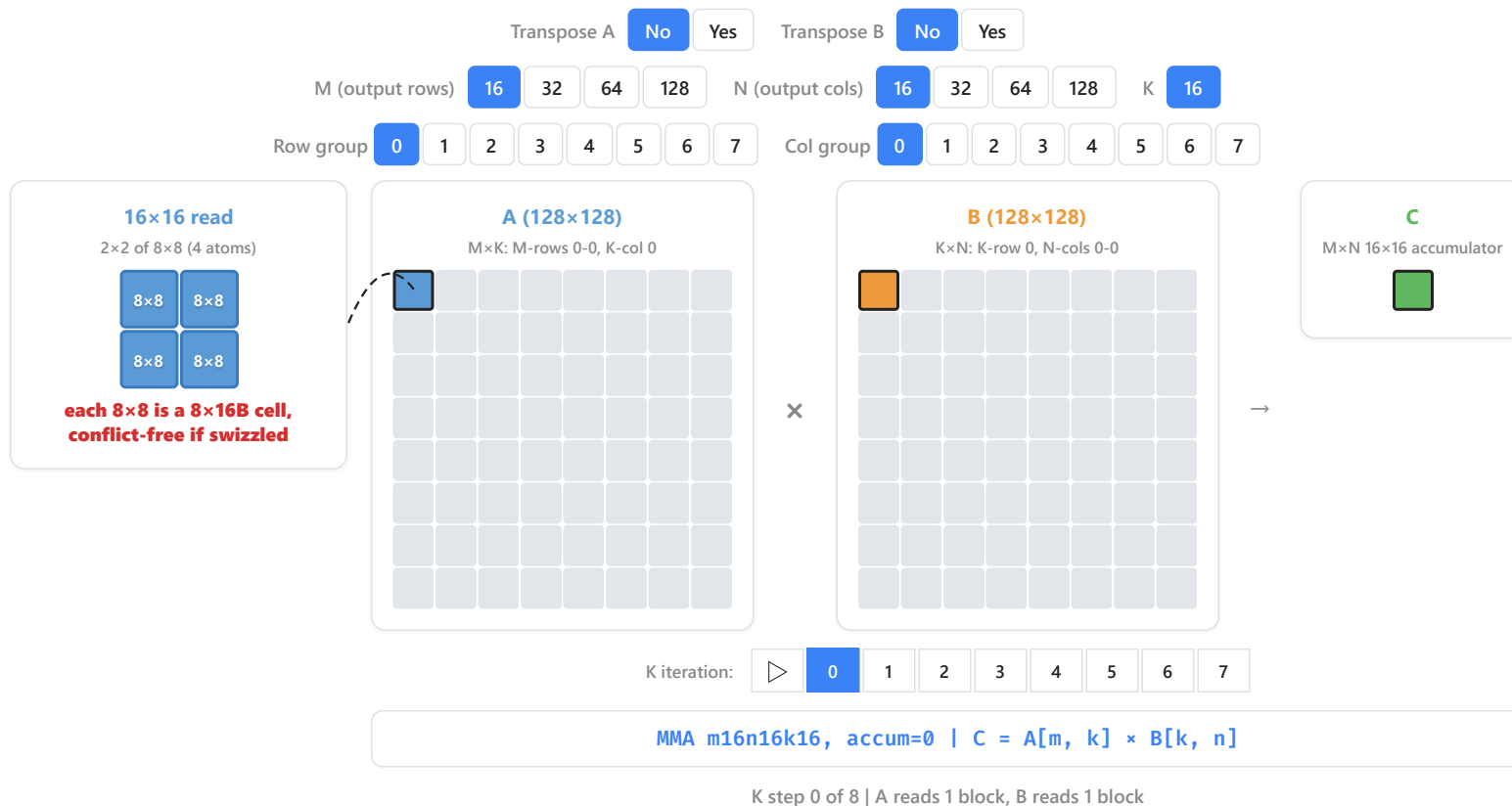




# Tensor Core Benefit From Swizzling

## Tensor Core Benefits From Swizzling

Tensor core reads  $M \times 16$  tile from shared memory – benefits from  $8 \times 8$  conflict-free read that swizzle guarantees





# Outline

Swizzle Deeper Dive

Tensor Cores

**TMA (Tensor Memory Accelerator)**

Putting It Together

# TMA: Global to Shared Memory

## TMA: Global to Shared Memory

Hardware 2D copy with optional swizzle – one instruction, no threads

Swizzle

None

128B

Row offset

0

8

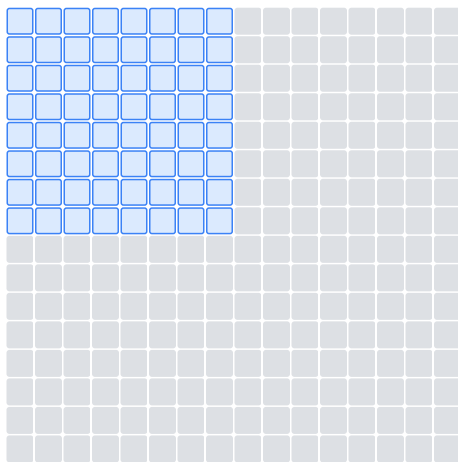
Col offset

0

64

### Global Memory

16×128 fp16 (each cell = 16B = 1×8 fp16)



### TMA

`cp.async.bulk.tensor.2d`  
(row=0, col=0)  
+ SWIZZLE\_128B

### Shared Memory

8×8 sectors (swizzled)

|    | s0        | s1        | s2        | s3        | s4        | s5        | s6        | s7         |
|----|-----------|-----------|-----------|-----------|-----------|-----------|-----------|------------|
| r0 | 0<br>@0   | 1<br>@16  | 2<br>@32  | 3<br>@48  | 4<br>@64  | 5<br>@80  | 6<br>@96  | 7<br>@112  |
| r1 | 1<br>@128 | 0<br>@144 | 3<br>@160 | 2<br>@176 | 5<br>@192 | 4<br>@208 | 7<br>@224 | 6<br>@240  |
| r2 | 2<br>@256 | 3<br>@272 | 0<br>@288 | 1<br>@304 | 6<br>@320 | 7<br>@336 | 4<br>@352 | 5<br>@368  |
| r3 | 3<br>@384 | 2<br>@400 | 1<br>@416 | 0<br>@432 | 7<br>@448 | 6<br>@464 | 5<br>@480 | 4<br>@496  |
| r4 | 4<br>@512 | 5<br>@528 | 6<br>@544 | 7<br>@560 | 0<br>@576 | 1<br>@592 | 2<br>@608 | 3<br>@624  |
| r5 | 5<br>@640 | 4<br>@656 | 7<br>@672 | 6<br>@688 | 1<br>@704 | 0<br>@720 | 3<br>@736 | 2<br>@752  |
| r6 | 6<br>@768 | 7<br>@784 | 4<br>@800 | 5<br>@816 | 2<br>@832 | 3<br>@848 | 0<br>@864 | 1<br>@880  |
| r7 | 7<br>@896 | 6<br>@912 | 5<br>@928 | 4<br>@944 | 3<br>@960 | 2<br>@976 | 1<br>@992 | 0<br>@1008 |

**TensorMap:** `dtype = f16, globalDim = [128, 16], globalStrides = [256B], boxDim = [64, 8], swizzle = CU_TENSOR_MAP_SWIZZLE_128B`

**Instruction:** `cp.async.bulk.tensor.2d.shared::cta.global.mbarrier::complete_tx::bytes [smem], [tensormap, {0, 0}], [mbar]`

**Note:** TensorMap and PTX follows column-major per official spec — globalDim, globalStrides, boxDim, and copy instruction coordinates are reversed from row-major order.

■ sector 0 ■ sector 1 ■ sector 2 ■ sector 3 ■ sector 4 ■ sector 5 ■ sector 6 ■ sector 7 | Cell value = logical col index | @N = smem byte offset





# 3D TMA for Tiling and Swizzling

## Tiling and Swizzling with 3D TMA

Use 3D TMA to copy into tiled shared memory

Swizzle

None

128B

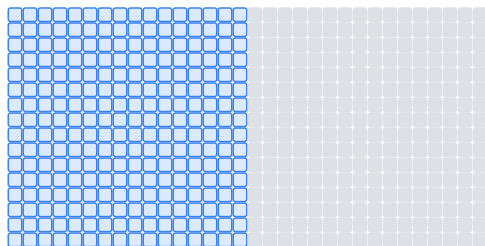
Col offset

0

128

### Global Memory

16×256 fp16 (contiguous, row = 512B)



### 3D TMA

cp.async.bulk.tensor.3d  
(row=0, group=0, col=0)  
+ SWIZZLE\_128B

### Shared Memory (tiled & swizzled)

g0

g1

|    | s0 | s1 | s2 | s3 | s4 | s5 | s6 | s7 |   | s0 | s1 | s2 | s3 | s4 | s5 | s6 | s7 |
|----|----|----|----|----|----|----|----|----|---|----|----|----|----|----|----|----|----|
| 0  | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 0 | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 0  |
| 1  | 1  | 0  | 3  | 2  | 5  | 4  | 7  | 6  | 1 | 0  | 3  | 2  | 5  | 4  | 7  | 6  | 1  |
| 2  | 2  | 3  | 0  | 1  | 6  | 7  | 4  | 5  | 2 | 3  | 0  | 1  | 6  | 7  | 4  | 5  | 2  |
| 3  | 3  | 2  | 1  | 0  | 7  | 6  | 5  | 4  | 3 | 2  | 1  | 0  | 7  | 6  | 5  | 4  | 3  |
| 4  | 4  | 5  | 6  | 7  | 0  | 1  | 2  | 3  | 4 | 5  | 6  | 7  | 0  | 1  | 2  | 3  | 4  |
| 5  | 5  | 4  | 7  | 6  | 1  | 0  | 3  | 2  | 5 | 4  | 7  | 6  | 1  | 0  | 3  | 2  | 5  |
| 6  | 6  | 7  | 4  | 5  | 2  | 3  | 0  | 1  | 6 | 7  | 4  | 5  | 2  | 3  | 0  | 1  | 6  |
| 7  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  | 7 | 6  | 5  | 4  | 3  | 2  | 1  | 0  | 7  |
| 8  | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 0 | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 0  |
| 9  | 1  | 0  | 3  | 2  | 5  | 4  | 7  | 6  | 1 | 0  | 3  | 2  | 5  | 4  | 7  | 6  | 1  |
| 10 | 2  | 3  | 0  | 1  | 6  | 7  | 4  | 5  | 2 | 3  | 0  | 1  | 6  | 7  | 4  | 5  | 2  |
| 11 | 3  | 2  | 1  | 0  | 7  | 6  | 5  | 4  | 3 | 2  | 1  | 0  | 7  | 6  | 5  | 4  | 3  |
| 12 | 4  | 5  | 6  | 7  | 0  | 1  | 2  | 3  | 4 | 5  | 6  | 7  | 0  | 1  | 2  | 3  | 4  |
| 13 | 5  | 4  | 7  | 6  | 1  | 0  | 3  | 2  | 5 | 4  | 7  | 6  | 1  | 0  | 3  | 2  | 5  |
| 14 | 6  | 7  | 4  | 5  | 2  | 3  | 0  | 1  | 6 | 7  | 4  | 5  | 2  | 3  | 0  | 1  | 6  |
| 15 | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  | 7 | 6  | 5  | 4  | 3  | 2  | 1  | 0  | 7  |

**TensorMap:** dtype = f16, globalDim = [64, 16, 4], globalStrides = [512B, 128B], boxDim = [64, 16, 2], swizzle = CU\_TENSOR\_MAP\_SWIZZLE\_128B

**Instruction:** cp.async.bulk.tensor.3d.shared::cta.global.mbarrier::complete\_tx::bytes [smem], [tensormap, {0, 0, 0}], [mbar]

**Key idea:** Let gslice be the 16×128 f16 matrix slice → gslice.reshape(16, 2, 64).transpose(1, 0, 2) has shape (2, 16, 64) → matches contiguous view of shared memory

**Note:** TensorMap and PTX follows column-major per official spec — globalDim, globalStrides, boxDim, and copy instruction coordinates are reversed from row-major order.

■ sector 0 ■ sector 1 ■ sector 2 ■ sector 3 ■ sector 4 ■ sector 5 ■ sector 6 ■ sector 7



# Outline

Swizzle Deeper Dive

Tensor Cores

TMA (Tensor Memory Accelerator)

Putting It Together

# Why Prefer 128B Swizzle?

## Why Prefer 128B Swizzle?

Each row of an atom is contiguous in global memory — wider rows = better locality

Highlight row

None

r0

r1

r2

r3

### SWIZZLE\_128B

8 rows × 8 cols = 8 × 128B

|    | c0 | c1 | c2 | c3 | c4 | c5 | c6 | c7 |
|----|----|----|----|----|----|----|----|----|
| r0 | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| r1 | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| r2 | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| r3 | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| r4 | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| r5 | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| r6 | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| r7 | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  |

**128B per row**

8 sectors × 16B = 128B contiguous

### SWIZZLE\_64B

8 rows × 4 cols = 8 × 64B

|    | c0 | c1 | c2 | c3 |
|----|----|----|----|----|
| r0 | 0  | 1  | 2  | 3  |
| r1 | 0  | 1  | 2  | 3  |
| r2 | 0  | 1  | 2  | 3  |
| r3 | 0  | 1  | 2  | 3  |
| r4 | 0  | 1  | 2  | 3  |
| r5 | 0  | 1  | 2  | 3  |
| r6 | 0  | 1  | 2  | 3  |
| r7 | 0  | 1  | 2  | 3  |

**64B per row**

4 sectors × 16B = 64B contiguous

### SWIZZLE\_32B

8 rows × 2 cols = 8 × 32B

|    | c0 | c1 |
|----|----|----|
| r0 | 0  | 1  |
| r1 | 0  | 1  |
| r2 | 0  | 1  |
| r3 | 0  | 1  |
| r4 | 0  | 1  |
| r5 | 0  | 1  |
| r6 | 0  | 1  |
| r7 | 0  | 1  |

**32B per row**

2 sectors × 16B = 32B contiguous

### SWIZZLE\_16B

8 rows × 1 col = 8 × 16B

|    | c0 |
|----|----|
| r0 | 0  |
| r1 | 0  |
| r2 | 0  |
| r3 | 0  |
| r4 | 0  |
| r5 | 0  |
| r6 | 0  |
| r7 | 0  |

**16B per row**

1 sector × 16B = 16B contiguous

**128B atom reads 128 contiguous bytes per row from global memory — 8× more locality than 16B interleaved**

**From tensor core side:** any swizzle format works (16×16 tile decomposes into any atom size).

**From global memory side:** each atom row is a contiguous segment. Wider row = fewer, larger reads = better bandwidth.

**SWIZZLE\_128B** gives 128 contiguous bytes per row — the best default unless tile width < 64 fp16.

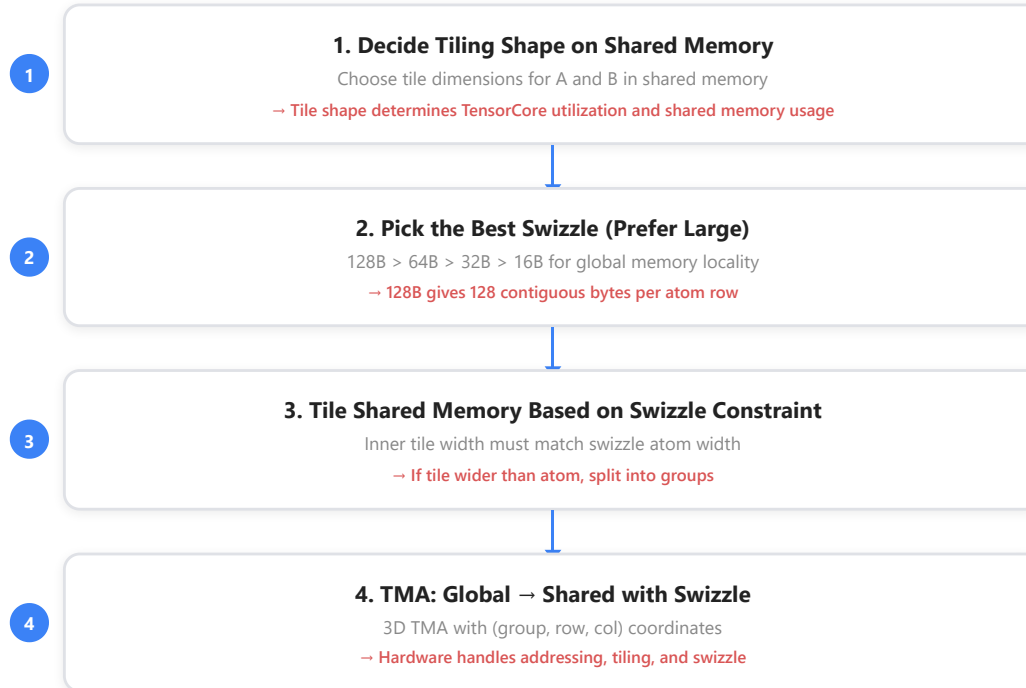
■ Sector 0 ■ Sector 1 ■ Sector 2 ■ Sector 3 ■ Sector 4 ■ Sector 5 ■ Sector 6 ■ Sector 7 | 1 cell = 16 bytes = 1 bank sector | Highlighted row = contiguous in global memory



# GEMM Optimization

## GEMM Optimization

From shared memory tiling to TMA data movement



[Click a step to see details](#)

Design shared memory layout first, then configure TMA to fill it efficiently.

Tiling shape → Best swizzle → Swizzle-constrained tiling → TMA with swizzle

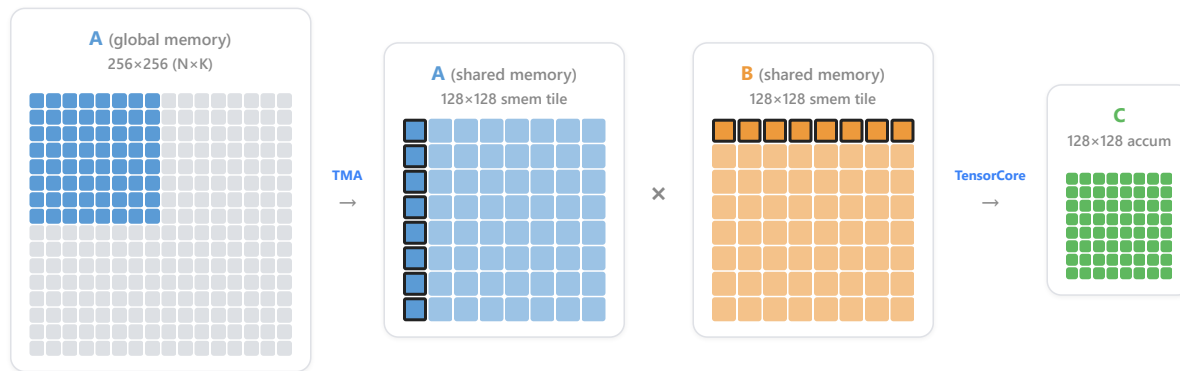




# End-to-End GEMM Example

## End-to-End GEMM: TMA + TensorCore

Data path: TMA loads tiles into shared memory, TensorCore reads for compute



▶ N block **0**  Outer K **0**  Inner K **0**

MMA m128n128k16, accum=0 | C = A[n, k] × B[k, m]

TMA: A[0:128, 0:128] → smem | TensorCore: K=0-15

N block 0/2 | K step 0 of 16 (outer K=0, inner K=0)

TensorCore read (current) | In smem (waiting) | In gmem | Consumed

B loading path (global → shared memory) omitted for simplicity

